{ backend }

Backend-программирование

Введение в курс

Java · HTTP · Spring Boot · PostgreSQL

Первая пара

Цели первой пары

- Понять, что именно мы будем называть backend
- Разобраться с форматом занятий и домашними работами
- Зафиксировать правила Git, Pull Request и проверки
- Подготовить окружение к следующему занятию

Короткая диагностика группы

Поднять руку или ответить в форме

- Кто уже писал программу больше одного файла?
- Кто пользовался терминалом и запускал проект из командной строки?
- Кто создавал ветку, делал commit, push и Pull Request?
- Кто понимает, что происходит после ввода адреса сайта в браузере?

Ответы не влияют на оценку. Они определят темп первых занятий.

Договорённости

Как общаемся, сдаём работу и получаем обратную связь

Коммуникация

- Общие объявления и быстрые вопросы: чат курса
- Персональные вопросы: почта университета
- Вопросы по конкретному коду: комментарии в Pull Request
- Если ответа нет больше семи дней, сообщение можно повторить с упоминанием преподавателя

Telegram



Вопрос по коду должен воспроизводиться

В сообщении нужны

- Ссылка на ветку или Pull Request с текущим состоянием
- Что ожидалось и что произошло фактически
- Полный текст ошибки и шаги для воспроизведения
- Что уже проверили самостоятельно

Скриншот без кода и фраза «не работает» не дают возможности помочь.

Как устроено обучение

- На занятии разбираем концепцию и собираем небольшой работающий пример
- После занятия материалы и код появляются в репозитории курса
- Домашняя работа закрепляет один конкретный навык
- Проверка проходит через ревью Pull Request

Нулевая работа: Git до backend

- Создать публичный репозиторий и заполнить README с ФИО и группой
- Создать отдельную ветку, добавить небольшой файл и сделать несколько осмысленных commit
- Отправить ветку на GitHub и открыть Pull Request в основную ветку
- Исправить замечание в том же Pull Request

Пока нулевая работа не принята, следующие ДЗ не проверяются.

Один репозиторий на студента

- Работы выполняются только в собственном публичном репозитории
- В README указаны ФИО, группа, способ запуска проекта и список выполненных заданий
- Основная ветка содержит только принятую или готовую к использованию версию
- Каждое задание разрабатывается в отдельной ветке

Одно задание — один Pull Request

- В Pull Request входят файлы текущего задания и необходимые изменения проекта
- Описание содержит номер задания, краткое решение и инструкцию запуска
- После возврата на доработку новый Pull Request создавать не нужно
- Merge выполняется только после принятия работы

Проверка и доработка

- Можно попросить принять работу сразу, если она уже соответствует минимальным требованиям
- Можно попросить оставить работу открытой, пока исправления заметно повышают качество
- После нескольких циклов ревью работа может быть принята при минимально допустимом качестве
- Повторяющиеся замечания в следующих работах влияют на оценку качества кода

Связанные задания сдаются последовательно

- Домашнюю работу лучше выполнять после занятия, к которому она относится
- Некоторые задания продолжают предыдущий проект
- Следующее связанное задание сдаётся после принятия предыдущего
- Несвязанные задания можно сдавать параллельно

Red flags при сдаче ДЗ

- Все работы отправлены в последние дни семестра
- Проект не собирается или не запускается по инструкции
- В Pull Request нет нужных файлов либо смешаны несколько заданий
- Замечания ревью игнорируются и повторяются в следующих работах
- В истории Git один commit с названием вроде final_final_2

Вопросы?

Сейчас лучше уточнить формат, чем переделывать сдачу

Критерии оценки

1. Работоспособность — 0–5

0 — не собирается 3 — основной сценарий работает 5 — все заявленные сценарии работают

2. Качество кода — 0–4

0 — нарушены базовые договорённости 2 — рабочая структура, много замечаний 4 — понятное решение

3. Бонус или штраф — –1…+1

За существенное улучшение, нарушение формата сдачи или систематическое игнорирование замечаний

Работа принята: итог ≥ 6

Просрочка: –1 балл за каждую неделю

Материалы курса

GitHub-репозиторий курса

https://github.com/WordyArc/dvfu-backend

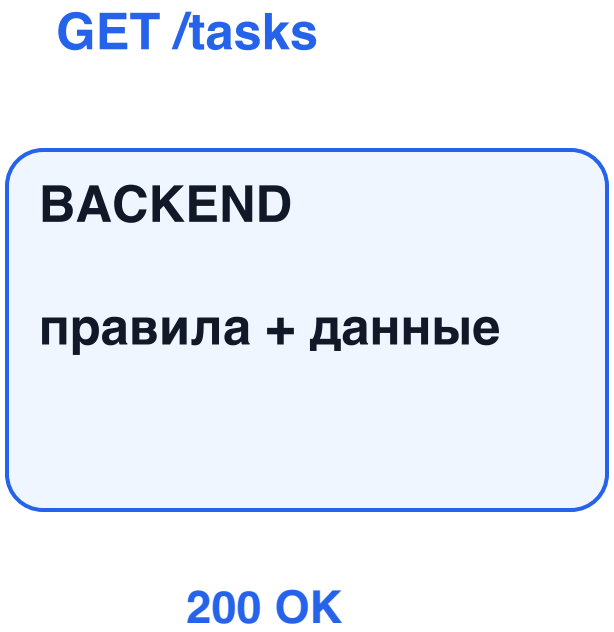
- Презентации и примеры с занятий
- Заготовки проектов и упражнения
- Описание домашних работ и критерии проверки

Backend

Часть системы, которая принимает запросы и отвечает за данные и правила

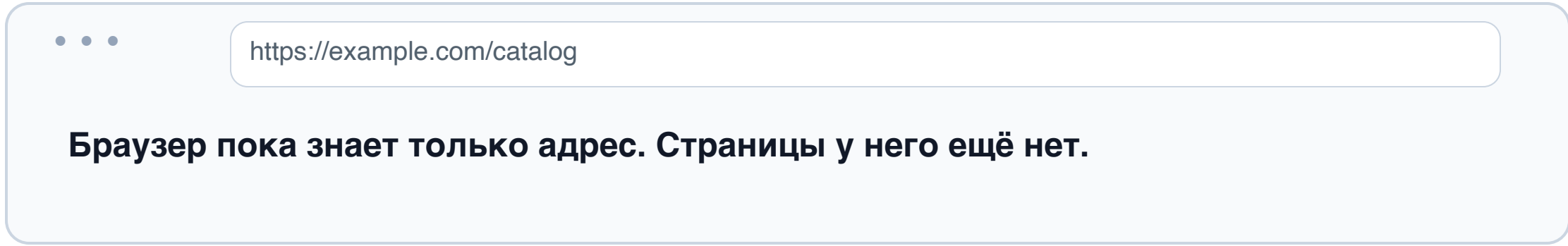
Что мы называем backend

- Программа, которая принимает запрос от клиента
- Проверяет данные и выполняет правила системы
- Работает с хранилищами и другими сервисами
- Возвращает клиенту результат или понятную ошибку



Клиентом может быть браузер, мобильное приложение или другая программа.

Открыли браузер и ввели адрес. Что дальше?



1

Находит сервер

DNS превращает имя сайта в IP-адрес

2

Устанавливает соединение

Клиент договаривается о защищённом канале

3

Отправляет запрос

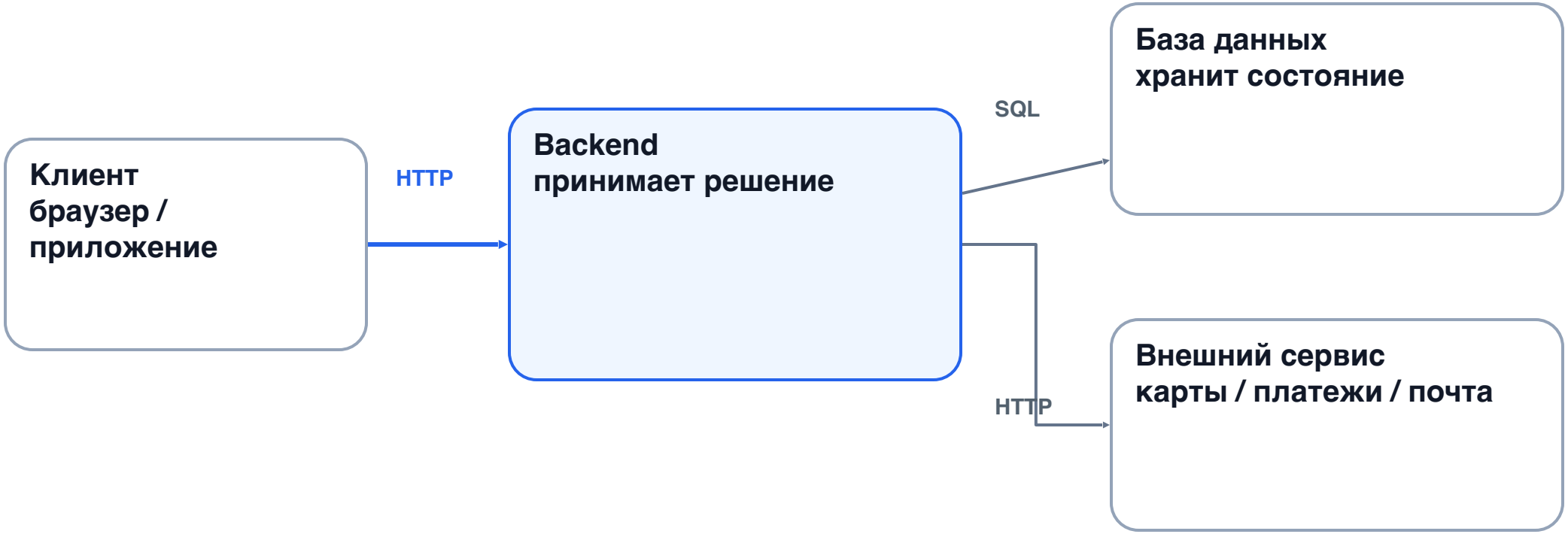
В запросе есть адрес ресурса и дополнительные данные

4

Получает ответ

Браузер показывает страницу или сообщение об ошибке

Кто с кем разговаривает



Backend связывает участников и скрывает внутреннее устройство системы от клиента.

Backend как контракт

ЗАПРОС

```
GET /catalog?page=2
Authorization: …
Accept: application/json
```

ОТВЕТ

```
200 OK
Content-Type: application/json
данные каталога
```

Контракт описывает, что клиент может попросить и какие ответы способен вернуть сервер.

- Frontend и backend могут разрабатываться отдельно
- Изменение контракта влияет на всех клиентов

Что обычно делает backend

AUTH

Понимает, кто пользователь и что ему разрешено

RULES

Применяет правила предметной области

DATA

Сохраняет и находит данные

API

Общается с клиентами и другими системами

Конкретный сервис может выполнять только часть этих задач.

Сервер, backend и база данных — разные вещи

01	**Сервер**	Компьютер или процесс, который принимает соединения
02	**Backend**	Приложение с правилами и обработкой запросов
03	**База данных**	Отдельная система для долговременного хранения

В учебном проекте все роли могут работать на одном ноутбуке. В большой системе они распределяются между множеством машин.

Почему backend быстро становится сложным

1

пользователь

10 000

одновременных пользователей

24/7

ожидание доступности

- Пользователь отправляет неверные или повторные данные
- Сеть, база данных или внешний сервис могут не ответить
- Два запроса меняют одни и те же данные одновременно
- Ошибки нельзя показывать вместе с секретами и внутренними деталями

Мини-задача: последнее место A7

МАША

нажимает «Купить»

A7

последнее свободное место

ПЕТЯ

нажимает «Купить»

Оба запроса приходят почти одновременно

Как backend должен продать место только одному человеку?

Раунд 1: спроектируйте покупку

01 Что клиент отправляет backend?

02 Что backend должен проверить?

03 Какие данные нужно сохранить?

04 Что получит каждый пользователь?

05 **Как избежать двойной продажи места?**

5 минут · результат можно нарисовать на листе или доске

Усложнение 1: клиент прислал цену

100 ₽

цена в запросе клиента

500 ₽

цена в системе

Пользователь изменил запрос перед отправкой

Какую цену должен использовать backend и почему?

Усложнение 2: нет ответа

СЦЕНАРИЙ А

Оплата прошла, но ответ потерялся. Пользователь снова нажал «Купить».

Как не списать деньги дважды?

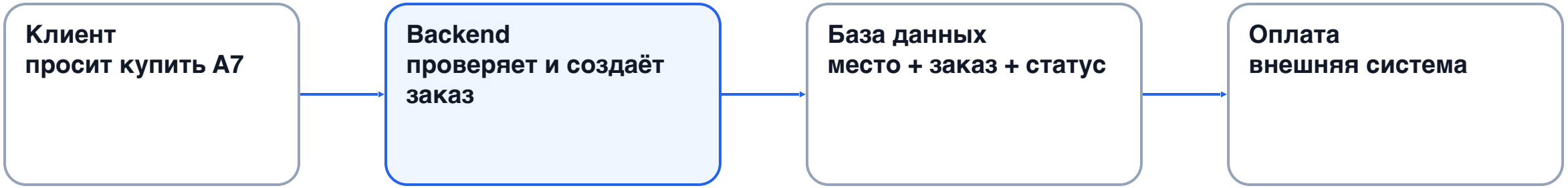
СЦЕНАРИЙ В

Платёжная система не отвечает. Неизвестно, прошла оплата или нет.

Что сохранить и что показать пользователю?

Backend должен помнить состояние операции и уметь безопасно продолжить её позже.

Разбор мини-задачи



- Backend проверяет пользователя, цену и доступность места
- База данных не должна разрешить две продажи одного места
- У операции есть идентификатор и сохранённый статус
- Неопределённый результат можно проверить и обработать позднее

Мы ещё не знаем конкретных механизмов. Их и будем изучать на курсе.

Технологии курса

- Java 21 — основной язык и базовые инструменты JVM
- HTTP и REST — сначала без магии фреймворка
- Spring Boot — когда понятны запрос, ответ и жизненный цикл приложения
- PostgreSQL — хранение данных и работа с SQL
- JUnit, Testcontainers и Docker — проверка и запуск сервиса

План курса

- Введение и Git
- Java для backend
- HTTP и устройство веб-приложения
- Spring Boot и REST API
- Валидация и обработка ошибок
- SQL и PostgreSQL
- Доступ к данным
- Тестирование
- Аутентификация и авторизация
- Интеграции и асинхронная работа
- Docker и конфигурация
- Итоговый сервис

Темп может меняться по результатам первых работ.

К следующему занятию

- Установить JDK 21, IntelliJ IDEA и Git
- Создать или проверить аккаунт GitHub
- Научиться выполнять clone, branch, add, commit, push
- Открыть первый Pull Request в рамках нулевой работы

Следующая тема: Git без магии и запуск первого Java-приложения

Вопросы?

Организация курса, Git, стек или содержание программы